

Schooltool Git-Workflow

Vollständige Bedienungsanleitung | Stand 22. September 2026

Mehrere Features, eine Vorschau: Unabhängige Arbeiten bleiben auf eigenen **feature/***-Branches offen. Sie wählen ausdrücklich, welches Feature die gemeinsame Online-Vorschau zeigt. Speichern auf GitHub, Vorschau, Freigabe nach main und Live-Deployment sind getrennte Schritte.

1. Die Arbeitsbereiche

Befehl / Schritt	Auswirkung und Voraussetzung
main	Gemeinsamer freigegebener Code. Erst gitdeploy aktualisiert daraus die Live-Anwendung; ein Push allein genügt nicht.
feature/matura feature/stundenplan feature/elternportal	Beispiele für drei unabhängige Arbeiten. Jeder Branch hat seine eigene unveränderliche Feature-Identität. Ein neues Feature beginnt vom aktuellen GitHub-main, nicht vom anderen Feature.
Lokaler Checkout	Ein Projektordner zeigt jeweils einen Branch. gitwork NAME wählt die Arbeit; gitsave sichert genau diesen Branch. Vor dem Wechsel Änderungen speichern.
Gemeinsame Online-Vorschau	Genau eine Installation mit eigener Datenbank, eigenen Dateien und Schlüsseln. gitpreview -Feature NAME wählt den zu veröffentlichenden Branch; er muss zum lokalen Checkout passen.
Live-Anwendung	Produktiver Datenbestand. Vorschau-Testdaten werden niemals nach Live übernommen. Live-Aktualisierung nur aus dem geprüften main-Release nach LIVE .

2. Welche Anleitung brauche ich gerade?

Nachschlagen	Inhalt
Seite 2	Windows-PC einrichten, Helfer aktualisieren, Voraussetzungen prüfen.
Seite 3	Mehrere Features anlegen, auswählen, speichern und Geräte wechseln.
Seite 4	Genau ein Feature online testen; Datenwechsel A zu B und zurück zu A.
Seite 5	Vorschau vorbereiten und fortsetzen; Zugriff und serverseitige Einrichtung.
Seite 6	Ein Feature freigeben, main-Prüfungen abwarten und live bereitstellen.
Seite 7	Abbrüche verstehen und sicher fortsetzen.
Seite 8	Täglicher Spickzettel, Versionierung und Befehlsübersicht.

Alle PC-Befehle in PowerShell im eigenen Schooltool-Projektordner ausführen. Namen in Beispielen durch die gewünschte Arbeit ersetzen. Befehle in Tabellen können aus Platzgründen umbrechen; jede dargestellte Befehlszeile einzeln eingeben.

Dieses Handbuch beschreibt die Bedienung der implementierten Funktionen. Es ersetzt weder den konkreten Installationsnachweis eines Servers noch die fachliche Abnahme eines Features. Zielwerte und geheime Zugangsdaten gehören in die geprüfte lokale Konfiguration, nicht in diese Anleitung.

Windows-PC einrichten

Einmal je Gerät; nach Workflow-Updates die aktuellen Helfer verwenden

1. Voraussetzungen

Benötigt werden Git und GitHub-Zugriff, PHP passend zu composer.lock mit PDO-MySQL und Sodium, Composer, Node/npm passend zum Projekt, PowerShell sowie Windows-OpenSSH. Für Live-Deployments muss die GitHub CLI **gh** angemeldet sein und die Actions des Projekt-Repositories lesen können.

Die lokalen Vollprüfungen benötigen derzeit MySQL unter **127.0.0.1:3306** mit dem lokalen Entwicklungskonto **root** ohne Kennwort. Der Testhelfer erstellt eigene zufällige Testdatenbanken und entfernt nur nachweislich selbst angelegte Datenbanken. Bei anderer lokaler Einrichtung muss der Helfer gezielt angepasst werden; keine Serverkonten dafür ändern.

2. Bestehenden sauberen main-Checkout aktualisieren

Befehl / Schritt	Auswirkung und Voraussetzung
<code>git status --short</code> <code>git branch --show-current</code>	Zuerst prüfen: keine ungesicherten Änderungen und tatsächlich main . Vorhandene lokale Arbeit zuerst sichern, nicht zurücksetzen.
<code>git pull --ff-only origin main</code>	Holt den veröffentlichten Workflow. Bei Divergenz stoppen und die unterschiedlichen Commits prüfen.
<code>composer setup:powershell</code>	Aktualisiert die eigenen Profile für Windows PowerShell und PowerShell 7. Danach ein neues Terminal im Projektordner öffnen.
<code>gitmain</code> <code>gitcheck</code>	Abhängigkeiten und Frontend vorbereiten und den Zustand kontrollieren. <code>gitcheck</code> zeigt auch offene Features, aber keinen Online-Serverstatus.

Ein neues Gerät erhält einen eigenen Clone und eine eigene lokale **.env**. Lokale Datenbanken, Uploads, private Schlüssel und der `.git`-Ordner eines anderen PCs werden nicht kopiert. Entwicklungsserver und Worker nach Branchwechsel neu starten.

3. Sichere Online-Verbindung

Für Hauptanwendung und Vorschau getrennte SSH-Schlüssel je PC verwenden. Die öffentlichen Schlüssel dem passenden Anwendungszugang zuordnen; private Schlüssel im lokalen Windows-SSH-Agent entsperren. Den Serverfingerabdruck unabhängig prüfen und in der vorgesehenen `known_hosts`-Datei hinterlegen. Die Helfer verlangen bekannte Hostschlüssel und Schlüsselauthentifizierung; sie reichen den Agent nicht an den Server weiter.

Geprüfte Ziel-, Pfad- und Schlüsselvariablen je Gerät einrichten. Die vollständigen Feldnamen und Befehle stehen im Projekt-README unter **SSH-Schlüssel und vertrauenswürdige Server je PC**. Keine Hostnamen oder Konten aus einem Beispiel übernehmen. Ein funktionierender SSH-Zugang beweist noch keine sichere Vorschau-Isolation.

4. Bestehende Features übernehmen

Alle verwendeten Feature-Branche brauchen den neuen Workflow-Code. Nach Aktualisierung von `main`: `gitwork NAME`, `gitupdate`, `testen` und `gitsave "Workflow übernommen"`. Falls das Terminal alte Definitionen hält, im Projekt `./scripts/git_helpers.ps1` laden.

Die bisherige Matura-Reservierung wird unverändert ihrem Branch zugeordnet. Ihre Identität und Vorschau-Datenbindung bleiben erhalten; eine manuelle Remote-Migration ist nicht nötig. Ältere Branches ohne Reservierung werden mit `gitwork NAME` gezielt registriert. Widersprüchliche Reservierungen stoppen den Ablauf.

Mehrere Features bearbeiten

Anlegen, auswählen, sichern und auf einem anderen Gerät fortsetzen

1. Zwei oder drei unabhängige Features beginnen

Befehl / Schritt	Auswirkung und Voraussetzung
<code>gitmain</code> <code>gitstart "stundenplan"</code>	Erstellt und reserviert feature/stundenplan vom aktuellen GitHub-main, pusht den Branch und wechselt dorthin. Ein bereits offenes Matura-Feature bleibt erhalten.
<code>gitsave "Erster Entwurf"</code>	Nach der Arbeit alle gewünschten Änderungen des aktuellen Branches committen und pushen. Zwischenstände sind erlaubt. Vorher gitcheck prüfen: gitsave nimmt alle Änderungen auf.
<code>gitmain</code> <code>gitstart "elternportal"</code>	Beginnt ein weiteres unabhängiges Feature aus main. Stundenplan und Matura bleiben offen. Weitere Features auf dieselbe Weise anlegen.
<code>gitsave "Portal begonnen"</code> <code>gitcheck</code>	Aktuellen Branch sichern und offene Features anzeigen. Ein erfolgreicher Push macht die Arbeit auf anderen Geräten verfügbar.

Namen bestehen aus Kleinbuchstaben, Ziffern und einzelnen Bindestrichen, z. B. **neue-funktion**. Ein bereits vorhandener Name wird mit gitwork fortgesetzt. gitstart erzeugt keinen zweiten Branch gleichen Namens.

2. Gezielt zwischen Arbeiten wechseln

Befehl / Schritt	Auswirkung und Voraussetzung
<code>gitwork "matura"</code>	Wählt feature/matura und bereitet Abhängigkeiten und Frontend vor. Andere Features bleiben offen. Ohne Namen ist gitwork nur bei genau einem offenen Feature eindeutig.
<code>gitsave "Matura-Zwischenstand"</code> <code>gitwork "stundenplan"</code>	Erst erfolgreich sichern, dann wechseln. Auf PC und Laptop identisch; am anderen Gerät denselben Namen angeben.
<code>gitsave "Zwischenstand"</code> <code>gitmain</code>	Zur gemeinsamen Basis wechseln, etwa um einen unabhängigen Fehler auf main zu beheben. Keine automatische Freigabe des Features.
<code>gitwork "stundenplan"</code> <code>gitupdate</code>	Aktuelles GitHub-main in den ausgewählten Feature-Branch übernehmen. Danach testen und mit gitsave sichern; kein automatisches Deployment.

3. Was Speichern bewirkt

Auf einem Feature: gitsave speichert ausschließlich dieses Feature auf GitHub. Es ändert keine Versionsnummer, schließt kein anderes Feature und aktualisiert weder Vorschau noch Live. Zum Online-Test danach ausdrücklich gitpreview aufrufen.

Auf main: gitsave baut den gebundenen Frontend-Release und pusht ihn. GitHub führt die erforderlichen Prüfungen aus. Erst anschließend kann gitdeploy diesen Stand live installieren. Einzelheiten und Versionierung stehen auf Seite 6.

Git schaltet keine Datenbank um. gitmain, gitwork und gitstart wechseln Code und bereiten Abhängigkeiten/Frontend vor, führen aber keine Migrationen oder Seeder aus. Bereits ausgeführte Migrationen bleiben bestehen. Für experimentelle Schemaänderungen eine eigene lokale Feature-Datenbank verwenden.

Ein Feature für die Vorschau

Die gemeinsame Vorschau zeigt immer genau die ausgewählte Arbeit

1. Auswählen, speichern und veröffentlichen

Befehl / Schritt	Auswirkung und Voraussetzung
gitwork "stundenplan" gitcheck	Zum gewünschten Feature wechseln. Ungesicherte Änderungen oder nur lokale Commits zuerst sichern.
gitsave "Vorschau vorbereitet"	Den gewünschten Stand vollständig auf GitHub speichern. Bei bereits sauberem, aktuellem Stand ist kein zusätzlicher Commit nötig.
gitpreview -Feature "stundenplan"	Als ein Befehl eingeben. Erstellt einen separaten Kandidaten mit aktuellem main und führt lokale Vollprüfungen aus. Bei mehreren Features ist -Feature ausdrücklich erforderlich; der Name muss zum Checkout passen.
REFRESH falls Daten ersetzt werden	Der Dialog zeigt den bevorstehenden Datenersatz. Beim Wechsel des bereits belegten Vorschau-Features ersetzt eine frische Live-Kopie die bisherigen aktiven Testdaten und Dateien. Nur bei gewünschtem Ersatz REFRESH eingeben.
PREVIEW	Den geprüften Vorschau-Stand ausdrücklich veröffentlichen. Der integrierte Feature-Stand wird auf GitHub und lokal übernommen und die Vorschau über SSH installiert. Andere Features bleiben unverändert.

Vor gitpreview ist gitupdate nicht nötig: Der Kandidat integriert main selbst. Die Vorschau ist kein automatischer Spiegel des zuletzt gespeicherten oder zuletzt geöffneten Branches.

2. Was geschieht mit den Testdaten?

Befehl / Schritt	Auswirkung und Voraussetzung
Erste Vorschau	Die erste Veröffentlichung erhält eine frische Live-Kopie. Es existiert noch kein Vorschau-Testbestand, der ersetzt werden müsste. PREVIEW bleibt erforderlich.
A erneut veröffentlichen	Bleibt dieselbe Feature-Identität aktiv, behalten normale Codeupdates die vorhandenen Testdaten. Vor Migrationen wird ein privater Wiederherstellungspunkt angelegt.
A zu B wechseln	Neue Live-Kopie nach REFRESH und PREVIEW . B übernimmt nicht die Teständerungen aus A.
B zurück zu A wechseln	Wieder eine frische Live-Kopie nach REFRESH und PREVIEW. Der frühere Teststand von A wird nicht wiederhergestellt.
gitpreview -Feature NAME -RefreshData	Als ein Befehl: Auch bei unverändertem Feature den aktiven Datenbestand ersetzen. Erfordert REFRESH und PREVIEW; vorher private Sicherung.

Es gibt keinen dauerhaften Testdatenstand je Feature. Ein später neu angelegtes Feature gleichen Namens hat eine neue Identität. Private Sicherungen dienen der gezielten Fehlerwiederherstellung, nicht dem normalen Wechsel zwischen Features.

Vor der Bestätigung prüfen: Ist das richtige Feature gewählt, und dürfen die bisherigen Teständerungen ersetzt werden? Der Status wird nach der Bestätigung und unter der Serversperre erneut geprüft. Ändert sich die Vorschau zwischenzeitlich, stoppt der Ablauf statt ungefragt einen anderen Datenstand zu ersetzen.

Vorschau sicher fortsetzen

Lokale Vorbereitung, Online-Zugang und einmalige Kompatibilität

1. Erst vorbereiten, später veröffentlichen

Befehl / Schritt	Auswirkung und Voraussetzung
<code>gitpreview prepare</code> <code>-Feature "stundenplan"</code>	Ein Befehl: Vollprüfungen und lokalen Kandidaten erstellen. Keine Änderung von GitHub-Branches oder Servern. Nicht mit <code>-RefreshData</code> kombinieren.
<code>gitpreview resume BUNDLE_ID</code> <code>-Feature "stundenplan"</code>	Ein Befehl: Den vom Helfer angezeigten Fortsetzungsbefehl mit der tatsächlichen Bundle-ID verwenden. Keine erneuten Tests bei unverändertem, gültigem Prüfnachweis. PREVIEW und nötigenfalls REFRESH bleiben erforderlich.
Abbruch bei Bestätigung	Leere oder falsche Eingabe veröffentlicht nichts. Der Helfer zeigt den Fortsetzungsbefehl. Kandidat, Paket und Nachweis aufbewahren.

Resume funktioniert nur auf demselben Windows-PC, unter demselben Benutzer und aus dem ursprünglichen Projektordner. Der geschützte Nachweis bindet Feature-Identität, Quellstand, main, Kandidat und Paket-Prüfsumme. Geänderte Branches, ungesicherte Arbeit oder ein ungültiger Nachweis stoppen die Fortsetzung. Bei neuem Quellstand wieder regulär `gitpreview` für vollständige Prüfungen verwenden.

Hat ein Veröffentlichungsversuch bereits begonnen, sperrt eine lokale **.started**-Markierung die automatische Wiederholung. Erst GitHub- und Serverzustand prüfen; diese Markierung nicht einfach löschen. Erhaltene Kandidaten in `git worktree list` sind Prüfarbeitsverzeichnisse, keine weiteren Nutzerfeatures.

2. Online-Zugriff und fachlich testen

Vorschau-Zugänge werden **in der Hauptanwendung** verwaltet. Jede Testperson braucht eine aktive, geeignete und ausdrücklich freigeschaltete Identität. Das gilt auch für Administrierende, Studierende sowie Eltern. Die Freigabe erweitert keine normalen Rechte. Das Superadmin-Kennwort erlaubt keine Anmeldung bei anderen Konten in der Vorschau.

Mit eigenen Zugangsdaten oder vorgesehenem Anmeldecode prüfen: Anmeldung, erlaubte und gesperrte Bereiche, Dateien und eine kleine Datenänderung. Kontrollierte Anmelde-Mails gesondert testen. Bei entzogener Freigabe oder fehlender Kontrollverbindung bleibt der Zugriff geschlossen. Eine technische Installation ersetzt keine fachliche Abnahme.

Die Live-Kopie enthält weiterhin personenbezogene Daten und bleibt geschützt. Aktive Live-Sitzungen, Reset-/Zugriffstoken und ausstehende Jobs werden nicht als aktive Vorschau-Zustände übernommen. Andere E-Mails und externe Integrationen sind gesperrt; nur zulässige Anmelde-/Passwort-Nachrichten dürfen versendet werden.

3. Server erstmals auf diesen Ablauf umstellen

Eine neue Vorschau benötigt die vollständige isolierte Einrichtung aus dem README: eigene Datenbank, Storage, Schlüssel, geschützte Snapshotablage und Kontrollverbindung zur Hauptanwendung. Kein `composer deploy` oder ungeprüftes `composer pdeploy` in der Vorschau verwenden.

Eine bereits isolierte ältere Vorschau braucht vor dem ersten neuen `gitpreview` einmalig die kompatible Status-/Planprüfung. Der geprüfte Einstieg ersetzt ausschließlich **FeaturePreviewSnapshotService.php** und **FeaturePreviewSnapshotCommand.php** aus dem Workflow-Release: private Übertragung und Sicherung, Vergleich der Ausgangshashes, PHP-Syntaxprüfung und Austausch unter derselben exklusiven Deployment-Sperre. Keine Änderung von Konfiguration, Datenbank, Uploads oder Feature-Identität.

Danach `preview:check` sowie `Snapshotstatus` und `assert-plan` für die bestehende Identität prüfen; `needs_snapshot` muss `false` bleiben. Anschließend den Workflow ins bestehende Feature übernehmen und regulär `gitpreview -Feature NAME` mit Vollprüfungen ausführen. Der normale Kandidat ersetzt den Übergangspatch. Den genauen kontrollierten Administratorablauf beschreibt das README unter der Vorschau-Kompatibilität; dies ist kein Schalter zum Umgehen regulärer Prüfungen.

Ein Feature freigeben und live

Andere offene Features und ihre Reservierungen bleiben erhalten

1. Genau das fertige Feature freigeben

Befehl / Schritt	Auswirkung und Voraussetzung
<code>gitwork "stundenplan"</code> <code>gitsave "Funktion fertig"</code>	Das fertige und fachlich geprüfte Feature auswählen und alle Änderungen sichern. Ein anderer offener Branch, etwa Matura, wird nicht mit freigegeben.
<code>gitrelease "Neuer Stundenplan"</code>	Prüft einen separaten Merge-Kandidaten mit aktuellem main. Bei Konflikten oder Fehlern stoppen und die Ursache im Feature beheben.
RELEASE	Nach erfolgreichen lokalen Vollprüfungen: geprüften Stand atomar nach GitHub-main veröffentlichen und nur diesen Feature-Branch und seine Reservierung schließen. Lokal zu main wechseln und vorbereiten.
<code>gitcheck</code>	Prüfen, dass main aktiv ist. Andere offene Features bleiben verfügbar. Release installiert noch nichts auf Live und wählt kein anderes Feature für die Vorschau.

Ändern andere Geräte main, Feature oder Reservierung während der Prüfung, schützen erneute Zustandsprüfungen und atomare Git-Vergleiche vor Überschreiben. Nicht mit Force-Push umgehen. Das Feature wird nur geschlossen, wenn genau der geprüfte Stand vollständig integriert ist.

2. main speichern und GitHub-Prüfungen abwarten

Bei direkter Arbeit auf main: `gitsave "Beschreibung"` bereitet Abhängigkeiten vor, prüft Format und Encoding, baut das Frontend und pusht den gebundenen Release. Die erforderlichen Tests laufen auf GitHub. Optional führt `gitsave "Beschreibung" -Full` zusätzlich lokale Vollprüfungen aus.

GitHub prüft Anwendungscode mit PHP-Tests/Coverage, Format und statischer Analyse, Frontend-Tests/Build sowie Linux- und nativen Windows-Workflowtests. Nur eindeutig reine Dokumentations-/Versionsänderungen dürfen unter strengen Herkunfts- und Inhaltsprüfungen einen verkürzten Weg nehmen. Eine neue Versionsnummer allein genügt nicht. Vorschau und Feature-Release behalten immer ihre lokalen Vollprüfungen.

3. Den exakt geprüften Release live installieren

Befehl / Schritt	Auswirkung und Voraussetzung
<code>gitdeploy</code>	Im sauberen Projekt ausführen. Prüft die erfolgreichen Pflichtjobs des exakten aktuellen GitHub-main-Release und zeigt das Live-Ziel. Fehlende, laufende oder gescheiterte Checks stoppen den Ablauf.
LIVE	Diesen Stand und dieses Ziel bestätigen. Nach erneuter Prüfung per SSH installieren, einschließlich vorgesehener Live-Datenbankschritte . Danach tatsächliche Erreichbarkeit und Fachfunktion prüfen.
<code>composer pdeploy</code>	Interner Serverpfad mit gebundenen Release-Identitäten und CI-Übergabe. Ein bloßer manueller Aufruf ist gesperrt; kein Ersatz bei fehlender GitHub-Freigabe.
<code>composer deploy</code>	Nur für das vollständige lokale Anwendungsupdate einschließlich lokaler Datenbankschritte. Vorher gitmain; auf einem Feature gesperrt.

Abbrüche und Fehler

Prüfen und sicher fortsetzen, ohne Daten oder fremde Arbeit zu verlieren

Befehl / Schritt	Auswirkung und Voraussetzung
Ungesicherte Änderungen oder lokale Commits	gitcheck prüfen und die gewünschte Arbeit mit gitsave erfolgreich sichern. Bei Konflikten gezielt auflösen. Kein blindes Reset oder Überschreiben.
Featureauswahl fehlt oder passt nicht	gitwork NAME verwenden, dann gitpreview -Feature NAME. Der Vorschau-Name muss zum aktuellen Checkout passen; die Auswahl allein wechselt keinen Branch.
GitHub-Stand geändert	Mit gitmain bzw. gitwork NAME aktualisieren. Echte Divergenz prüfen und auflösen. Kandidaten und Nachweise nicht auf einen anderen Stand umbiegen.
Tests oder Build scheitern	Fehler im betreffenden Quellstand beheben, gezielt testen, mit gitsave sichern und den Vorgang neu vollständig prüfen. Keine Test- oder CI-Sperren umgehen.
Preview-Plan geändert	Aktuellen Serverstatus und gewähltes Feature prüfen. Den Ablauf erneut planen und Datenersatz neu bestätigen; es wird nicht still weiterimportiert.
Release veröffentlicht, lokale Vorbereitung kaputt	Ursache beheben und gitmain wiederholen. Der lokale Feature-Branch bleibt zur Sicherheit erhalten. Nur nach erfolgreicher Prüfung den vollständig integrierten Branch nötigenfalls mit git branch -d feature/name entfernen; kein -D.
SSH/Deployment unterbrochen	GitHub- und tatsächlichen Serverzustand prüfen. Kandidaten, Paket, Nachweise und private Sicherungen erhalten. Nicht ungeprüft erneut veröffentlichen. Bei Vorschaufehlern bleibt der Zugang geschlossen.

Gezielte Vorschau-Wiederherstellung

Nur nach Prüfung **im Vorschau-Verzeichnis**: `php artisan preview:snapshot restore --replace --no-interaction`. Stellt ausschließlich den zum offenen Wiederherstellungsvorgang passenden privaten Daten-/Dateistand und Vorschau-Zustand wieder her. Code wird nicht zurückgerollt; die Vorschau bleibt geschlossen. Danach Ursache beheben und gitpreview für passende Code-/Migrationsprüfungen erneut ausführen. Marker nicht manuell löschen; bei fehlender Sicherung oder fehlendem Schlüssel keinen Import erzwingen.

Keine automatische Rückabwicklung

Ein Fehler bedeutet nicht automatisch, dass zuvor erfolgreiche Git- oder Datenbankschritte rückgängig gemacht wurden. Ermitteln Sie zuerst den tatsächlich erreichten Zustand. Die erhaltenen Kandidaten und Sicherungen unterstützen eine gezielte Wiederherstellung; sie sind kein Auftrag zum ungeprüften Wiederholen.

Bei laufenden oder fehlgeschlagenen GitHub-Prüfungen nicht auf den manuellen Serverbefehl ausweichen. Ein neues gitsave erzeugt einen neuen Release-Stand und braucht dessen eigene erfolgreiche Prüfungen. Die Freigabe eines älteren Commits gilt nicht für einen neueren.

Der tägliche Spickzettel

Befehle in der angegebenen Reihenfolge einzeln ausführen

Anlass	Reihenfolge (Befehle einzeln ausführen)
Weitere Arbeit anfangen	gitmain; dann gitstart "name"; entwickeln; gitsave "Beschreibung".
Arbeit / Gerät wechseln	Erfolgreich gitsave; am Zielgerät gitwork "name" oder gitmain.
main ins Feature holen	gitwork "name"; gitupdate; testen; gitsave "main übernommen".
Ausgewählte Arbeit testen	gitwork "name"; sichern; gitpreview -Feature "name"; nötigenfalls REFRESH, dann PREVIEW.
Nur ein Feature freigeben	gitwork "name"; sichern; gitrelease "Beschreibung"; RELEASE. Andere Features bleiben offen.
Live aktualisieren	Erfolgreiche GitHub-Pflichtprüfungen des exakten main-Release; gitdeploy; LIVE; Anwendung prüfen.

Optional eine Versionsnummer vergeben

gitrelease "Neue Funktion" "3.49.0" bzw. auf main gitsave "Beschreibung" "3.49.0". Nummer ohne v; Beispiel durch die gewünschte Version ersetzen. Passende Notizen in UPDATES.md und das lokale Docusaurus-Projekt müssen vorbereitet sein. Der Ablauf aktualisiert Version, Changelog, Dokumentation und Versions-Tag. Feature-gitsave nimmt keine Version an.

Das Docusaurus-Projekt liegt standardmäßig unter C:/docusaurus/schooltool; ein anderer Pfad wird mit SCHOOLTOOL_DOCUMENTATION_ROOT konfiguriert. Eine Versionsnummer ist für normale Zwischenstände und die reine Workflow-Einrichtung nicht erforderlich.

Kompatibilitätsbefehle

gitpush bleibt auf main kompatibel zu gitsave; -Full ergänzt auch dort lokale Vollprüfungen. Im Alltag genügt gitsave. gitget, gitcommit und gitmerge sind für den beschriebenen Ablauf nicht nötig.

Die vier Bestätigungen: REFRESH erlaubt den angezeigten Ersatz der Vorschau-Testdaten; PREVIEW veröffentlicht den geprüften Vorschau-Kandidaten; RELEASE gibt genau das ausgewählte Feature nach main frei; LIVE installiert den exakt geprüften main-Release auf der Hauptanwendung.

Quellen und Dokumentpflege

Weiterführende Einrichtung: README.md im gleichen Projektstand. PDF-Quelle: scripts/build_git_workflow_pdf.py; Erzeugung mit Python und ReportLab sowie Windows-Schriften Arial/Consolas. Die Anleitung ist bewusst ohne geheime Zielkonfiguration und ohne pauschale Server-Abnahmebestätigung.